

CRUSTPairs: A Cross-Language Dataset of Paired C and Rust Solutions

Leonardo J. Almeida
University of São Paulo
leonardo.j.almeida@usp.br

Vojtěch Hejlek
TODO affiliation
TODO email

TABLE I

CROSS-LANGUAGE CODE CORPORA. CRUSTPAIRS IS THE ONLY ONE PAIRING C WITH RUST AT SOLUTION GRANULARITY. TODO: VERIFY EVERY CELL.

Dataset	Languages	Paired	Rust
CodeNet	55	no	no
xCodeEval	17	yes	no
XLCoST	7	yes	no
CoST	7	yes	no
AVATAR	2	yes	no
CRUSTPairs	2	yes	yes

Abstract—C-to-Rust migration is an active agenda, but no dataset pairs the two languages at the level of whole solutions to a shared problem. We present CRUSTPairs, 1857 C/Rust implementation pairs drawn from competitive programming archives, normalised, deduplicated, and stratified into difficulty tiers by zero-shot cross-language embedding similarity, with per-record provenance and licensing. Cross-language retrieval is hard on this data, and harder than a single baseline would suggest: TF-IDF over shared identifiers reaches MRR 0.21 and so beats a pretrained code model at 0.19, which on the hardest quarter of the data scores $1.7\times$ chance. Finetuning reaches 0.79, and lifts that hardest quarter $10\times$. Renaming every user identifier settles what is being learned: the finetuned models keep 97% of their accuracy without them, while the pretrained model keeps 66% — less than the lexical baseline does — and on structurally divergent pairs the finetuned models are *better* without identifiers than with them.

Index Terms—dataset, cross-language, code similarity, Rust, C, code retrieval

I. INTRODUCTION

Contributions.

- CRUSTPairs, 1857 C/Rust solution pairs with problem statements, normalised entry points, per-record provenance and licensing.
- A difficulty stratification derived from zero-shot cross-language embedding similarity, independent of the model family used in evaluation.
- Retrieval baselines, per difficulty tier, over a frozen stratified split.

II. RELATED DATASETS

Table I places CRUSTPairs among them.

III. CONSTRUCTION

A. Sources and pairing

CRUSTPairs draws on two competitive programming archives: xCodeEval [1], itself derived from Codeforces, and IBM Project CodeNet [2], derived from AtCoder. Both distribute accepted submissions, so the two implementations of a pair are known to have passed the same official tests. A pair is one problem, one accepted C submission, and one accepted Rust submission.

B. Normalisation

Both sides pass through a static analysis pipeline that strips comments, removes unused functions and macros, normalises encoding, and applies the language’s standard formatter (`clang-format`, `rustfmt`). Entry points are renamed to `solution` in both languages so that the function name carries no signal. One consequence, discussed in Section VII, is that the distributed code is no longer a standalone compilable program.

C. Cleaning

From an initial 2013 pairs, 127 were removed: 2 confirmed mis-pairs, 69 AtCoder easy/hard variants whose statements were near-identical (description embedding similarity > 0.90), 54 further suspicious pairs (0.80–0.90), 1 length ratio outlier, and 1 case of genuine implementation divergence. Near-duplicate detection used OpenAI `text-embedding-3-large` over the raw statements.

A further 29 pairs, a set of textbook reference implementations, were dropped for two converging reasons. They carried the dataset’s only copyleft content and were its only records whose two halves were under different licences (Section VIII); and they were structurally unlike the rest, having no problem statement and functions taking arguments rather than reading standard input. This leaves 1857 pairs.

D. Difficulty stratification

Each pair is scored by the cosine similarity between zero-shot `SFR-Embedding-Code-400M_R` embeddings of its C and Rust sides, and binned at the 25th and 75th percentiles into easy, medium and hard (thresholds 0.7856 and 0.8687; similarity mean 0.826, standard deviation 0.058). The raw score is distributed alongside the tier, so the cuts can be redrawn without re-running the model. Pairs a generic code

TABLE II
SIZE OF THE TWO SIDES, IN CHARACTERS AND LINES.

	Mean	Median	p90	Max
C, characters	616	516	1046	16460
Rust, characters	851	749	1424	4888
Statement, chars	1459	1231	2717	7368
C, lines	33.6			
Rust, lines	31.4			

TABLE III
COMPOSITION BY SOURCE AND DIFFICULTY TIER.

Source	Easy	Medium	Hard	Total
CodeNet (AtCoder)	241	408	190	839
xCodeEval (Codeforces)	224	520	274	1018
Total	465	928	464	1857

model already embeds close together are near-literal translations; pairs it separates diverge structurally and demand a real cross-language remapping.

The choice of SFR rather than UniXcoder [3] is deliberate. Defining the tiers with the same model family that is later finetuned would make hard pairs hard by construction, and any reported per-tier improvement would be circular. SFR is trained on a different corpus, so the tiers remain an independent axis. The stratification is also origin-agnostic: AtCoder and Codeforces publish difficulty ratings on incompatible scales, so upstream ratings are not used.

IV. DATASET DESCRIPTION

A. Schema

B. Statistics

Table II gives the size of each side. The Rust side is about $1.5\times$ longer than the C side in characters (median ratio 1.50) while being *shorter* in lines (31.4 against 33.6): Rust solutions pack more into each line. This is worth stating because it means a length-based heuristic does not separate the two languages the way a naive reading of the character counts would suggest.

Table III breaks the dataset down by both axes. Source and difficulty are not independent: the CodeNet portion skews easy while the xCodeEval portion skews hard. Reporting a per-tier result therefore requires a split that controls for both.

C. Splits

The train/validation/test assignment is frozen in the data rather than regenerated from a seed, and is stratified by source $\times$ difficulty at 80/10/10: 1485, 185 and 187 pairs. The test set mirrors the full dataset to within 0.3 percentage points by source and 0.6 by tier.

V. BASELINES

Retrieval is C-to-Rust unless stated otherwise: each C implementation queries the pool of all 187 Rust implementations in the test split, and the target is the one solving the same

TABLE IV
CROSS-LANGUAGE RETRIEVAL ON THE 187-PAIR TEST SET.

Model	MRR	MRR@10	R@1	R@5	R@10
Chance	0.031	0.016	0.005	0.027	0.053
Lexical TF-IDF	0.206	0.191	0.128	0.273	0.353
UniXcoder, zero-shot	0.189	0.170	0.134	0.203	0.283
UniXcoder + LoRA	0.724	0.720	0.624	0.836	0.913
UniXcoder, full	0.792	0.791	0.706	0.909	0.952

TABLE V
MRR BY DIFFICULTY TIER, WITH AND WITHOUT USER IDENTIFIERS. EVERY TIER IS SCORED AGAINST THE SAME 187-CANDIDATE POOL; ONLY THE QUERIES ARE PARTITIONED. *Ret.* IS THE ANONYMISED MRR AS A FRACTION OF THE NORMAL ONE.

Model	Ident.	Easy	Medium	Hard	Ret.
Lexical TF-IDF	kept	0.446	0.144	0.089	76%
	anon	0.355	0.094	0.080	
UniXcoder, zero-shot	kept	0.328	0.186	0.053	66%
	anon	0.276	0.084	0.048	
UniXcoder + LoRA	kept	0.957	0.776	0.388	96%
	anon	0.941	0.688	0.429	
UniXcoder, full	kept	0.979	0.821	0.550	97%
	anon	0.951	0.756	0.611	

problem. Chance is $\text{MRR} = 0.031$ at this pool size. LoRA figures are mean $\pm$ standard deviation over four seeds; the other rows are single runs.

The lexical row is the one that fixes the frame. TF-IDF over shared identifiers, with no model at all, reaches 0.206 and so *beats* zero-shot UniXcoder [3] at 0.189. Competitive programmers reuse variable names across languages: `n` is shared by 85 of the 187 test pairs, `i` by 84, then `a`, `t`, `s`, `j`. The claim this dataset supports is therefore not that a generic code model finds the task hard, but the stronger one that *neither* generic code embeddings *nor* lexical matching solve it, and finetuning does — $3.8\times$ over zero-shot for LoRA [4] and $4.2\times$ for full finetuning ($4.2\times$ and $4.7\times$ on MRR@10, which diverges from MRR because many zero-shot positives fall outside the top ten).

A. Difficulty tiers and the identifier ablation

Table V reports MRR per tier, and repeats every configuration with user identifiers renamed to positional names independently in each language, which destroys cross-language name correspondence while leaving control flow, types, library calls and string literals intact.

Three things follow, and together they are the strongest evidence that this dataset measures what it is meant to.

Retrieval is monotone in tier for every configuration. The tiers were cut with SFR while the models here are UniXcoder-based, so the ordering is a property of the pairs rather than an artefact of the encoder that defined it. Zero-shot UniXcoder reaches 0.053 on the hard tier — $1.7\times$ chance, with exactly one of the 47 hard pairs at rank 1. A generic code model effectively cannot solve a quarter of this dataset.

Finetuning learns structure, not names. The finetuned models keep 96–97% of their MRR when identifiers are destroyed; zero-shot UniXcoder keeps 66%, which is *less* than the 76% the lexical baseline keeps. A pretrained code model is more dependent on variable names than TF-IDF is. That inverts the natural reading of Table IV: zero-shot UniXcoder does not merely fail to beat lexical matching, it is doing a worse version of the same thing.

On hard pairs, identifiers are a liability. Both finetuned models do *better* without them — LoRA from 0.388 to 0.429, full finetuning from 0.550 to 0.611, both about +11% — while the lexical baseline and the zero-shot model get worse. Where implementations diverge structurally, a shared name is more often a coincidence than a signal, and a model trained without that cue learns the remapping instead of leaning on it.

B. Retrieval direction

Querying with Rust is consistently easier than querying with C: MRR rises from 0.189 to 0.233 zero-shot and from 0.792 to 0.836 with full finetuning. Under the ablation the asymmetry all but disappears for the finetuned models (0.768 against 0.773), which locates its cause: the Rust side carries more of the identifier signal, not more of the structure.

C. Parameter efficiency depends on difficulty

LoRA reaches 91% of full finetuning overall while updating 0.23% of the parameters, which invites the conclusion that the adapter is nearly free. Per tier it is not: LoRA recovers 98% of full performance on easy pairs, 95% on medium, and 71% on hard. The parameter budget binds precisely where the cross-language remapping is non-trivial, and a single aggregate ratio conceals that.

VI. USE CASES

A. Why C-to-Rust is under pressure

Memory-safety defects are the dominant vulnerability class in C codebases of record. Microsoft’s Security Response Center has for two decades put roughly 70% of the company’s CVE-assigned vulnerabilities in that class, a figure its Azure CTO restated in 2025 [5]; Microsoft has been rewriting core Windows libraries in Rust since 2023 and designated Rust a first-class internal language in 2026. Android gives the clearest evidence that the substitution works: annual memory-safety vulnerabilities fell from 223 in 2019 to under 50 in 2024 and dropped below 20% of Android’s total for the first time, with Google reporting roughly a 1000× lower memory-safety defect density in its Rust code than in its C and C++ [6]. Policy has followed the evidence, through CISA’s memory-safe roadmaps [7] and the US Office of the National Cyber Director’s 2024 call to adopt memory-safe languages [8].

The resulting problem is translation at scale, and it is explicitly a research programme rather than an engineering task. DARPA’s TRACTOR — Translating All C to Rust — launched in July 2024 to automate the conversion of legacy C to idiomatic Rust, on the premise that two decades of bug-finding tools have not been enough and that the Department of Defense’s C-based systems are too large to port by hand [9].

B. What a paired corpus contributes

Translation tools are evaluated almost entirely on whether their output compiles and passes the original tests. TRACTOR’s own benchmark is a battery of 150 C programs scored for correctness and safety [10]. That is the right primary criterion and an incomplete one: a translation can pass every test and still be an unfaithful or unidiomatic rendering of the source, and a test suite is exactly what legacy C most often lacks. The complementary question — *are these two programs, in two languages, the same program?* — is a similarity judgement, and it is the one CRUSTPairs supplies supervision for.

Four concrete uses follow.

Scoring translator output without tests. A model finetuned on CRUSTPairs assigns a cross-language similarity to a (C, Rust) pair. Applied to a translator’s input and output it gives a test-free signal that flags translations drifting from their source — usable as a regression check across tool versions, or to rank candidates when a tool emits several. The per-tier results in Section V matter here: the usable range of such a score is set by how well a model does on *structurally divergent* pairs, which is where the hard tier shows the current ceiling is 0.55 MRR, not the 0.98 the easy tier would suggest.

Finding an existing port. Migrating a C codebase rarely starts from nothing: equivalent functionality often already exists in Rust, in a crate or in a sibling project. Locating it is cross-language retrieval over whole implementations, which is the task this dataset measures directly.

Cross-language clone and vulnerability tracking. When a CVE is filed against a C implementation, the question of whether a Rust port inherits the defect — or whether a Rust crate is a translation of vulnerable C — is a type-4 clone problem across languages. Type-4 clone corpora are overwhelmingly single-language; a paired C/Rust corpus gives the cross-language case supervision it has lacked.

Studying what ownership changes. Each pair is two solutions to one problem, one written with a borrow checker and one without. The dataset is therefore a controlled sample for asking how ownership and lifetimes reshape an implementation — a question the difficulty tiers already index, since the hard tier is precisely the subset where the two languages’ solutions diverge structurally rather than translating line for line.

C. What it does not support

The pairs are competitive programming solutions: short, single-file, I/O-driven, and free of the allocation patterns, pointer aliasing and concurrency that make real C hard to port. Results here should be read as a measurement on a clean proxy for the cross-language matching problem, not as evidence about systems code, which is where the migration pressure described above actually sits. Section VII returns to this.

VII. LIMITATIONS

Equivalence is inherited, not verified. No test cases or expected I/O are distributed. That both submissions were accepted upstream is evidence of functional equivalence, but this dataset does not itself execute anything, and the entry-point renaming means the distributed code does not compile as a standalone program. Execution-based verification is the clearest line of future work.

Upstream identifiers are absent. The pairs cannot be traced back to the originating problem or submission, which limits attribution to the corpus level and prevents joining against upstream metadata.

Competitive programming is not production code. Solutions are short, single-file, and I/O-driven. Conclusions drawn here should not be assumed to transfer to systems code, which is where C-to-Rust migration actually matters.

Difficulty is a proxy. The tiers measure embedding similarity, not algorithmic difficulty or translation effort.

Variance is measured for one configuration only. The LoRA rows are means over four seeds; the zero-shot, full-finetune and lexical rows are single runs. The LoRA spread ($\sigma = 0.011$ on MRR) is the only direct evidence available for how much of any gap here is seed noise.

The original tier labels were not exactly reproducible. An earlier release of this dataset shipped tier labels whose generating script had since diverged. Regenerating them reproduces the aggregate distribution closely (mean 0.826 against a reported 0.823) but agrees on only 89% of individual pairs, with the disagreement spread symmetrically across adjacent tiers — a pattern that threshold movement alone cannot produce, and that indicates the per-pair scores differed. The labels released here are the regenerated ones, and the raw scores are now distributed so that the question cannot recur.

Pre-training contamination is possible. Both source archives are public and may appear in the pre-training corpora of the models evaluated here.

VIII. AVAILABILITY AND LICENSING

CRUSTPairs aggregates two corpora under different licences, so it is not distributed under a single one; every record carries its own SPDX expression. The 1018 xCodeEval-derived pairs are CC BY-NC 4.0 and the 839 CodeNet-derived pairs are CDLA-Permissive-2.0. The aggregate is therefore non-commercial, and attribution to both upstream projects is required. Filtering to the CodeNet portion yields a 839-pair subset with no commercial restriction.

ACKNOWLEDGEMENTS

TODO.

REFERENCES

- [1] M. A. M. Khan, M. S. Bari, D. X. Long, W. Wang, M. R. Parvez, and S. Joty, “xCodeEval: An execution-based large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval,” in *TODO*, 2023.
- [2] R. Puri, D. S. Kung, G. Janssen, W. Zhang, G. Domeniconi, V. Zolotov, J. Dolby, J. Chen, M. Choudhury, L. Decker, V. Thost, L. Buratti, S. Pujar, S. Ramji, U. Finkler, S. Malaika, and F. Reiss, “CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [3] D. Guo, S. Lu, N. Duan, J. Yin, D. Jiang, and M. Zhou, “UniXcoder: Unified cross-modal pre-training for code representation,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.
- [4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [5] M. Russinovich, “Keynote,” RustConf, 2025, reports that roughly 70% of Windows CVEs are memory-safety issues.
- [6] Google, “Rust in Android: Move fast and fix things,” Google Security Blog, 2025, android memory-safety vulnerabilities fell from 223 in 2019 to under 50 in 2024, below 20% of the total.
- [7] Cybersecurity and Infrastructure Security Agency, “The case for memory safe roadmaps,” Dec. 2023.
- [8] Office of the National Cyber Director, “Back to the building blocks: A path toward secure and measurable software,” The White House, Feb. 2024.
- [9] Defense Advanced Research Projects Agency, “Eliminating memory safety vulnerabilities once and for all,” DARPA news release, TRACTOR programme (Translating All C to Rust), Jul. 2024. [Online]. Available: <https://www.darpa.mil/news/2024/memory-safety-vulnerabilities>
- [10] MIT Lincoln Laboratory, “Translating all C to Rust (TRACTOR) benchmarks,” Project page. [Online]. Available: <https://www.ll.mit.edu/r-d/projects/translating-all-c-rust-tractor-benchmarks>